

JEWELRY TRADER

ESTATE & FINE JEWELRY · POINT OF SALE, INVENTORY & E-COMMERCE

Phase 1 Application Handbook

How the system works, what it does, and how to run it — written against the delivered build, not the plan.

12

MODULES DELIVERED

316

TESTS PASSING

47

DATABASE TABLES

34

PERMISSIONS

Prepared 23 September 2026 · Laravel 13 · PHP 8.3 · MySQL 8 · Stripe verified in test mode

What this application is

One system covering the whole life of a piece: intake and appraisal, the shop counter, the public storefront, and the money that follows — wages, refunds and the audit trail behind them.

Jewelry Trader serves dealers in antique, vintage, estate, designer and fine jewelry, across several store locations and several salespeople. A piece is photographed and catalogued once, then travels through review, listing, sale and — if it comes back — return, with every step recorded against the person who took it.

The defining characteristic

Almost nothing is hardcoded. Payment providers, commission structures, tax rates, required-field rules, discount ceilings, retention windows and feature availability are all configured from the admin panel by a Super Admin. No code change, no redeploy.

What that means in practice	
Stripe keys	Stored encrypted in the database, rotated through a form. Not in .env.
Which fields are mandatory	A table of rules, editable per item category. A ring can require a size; a brooch cannot.
Commission structure	Four plan types, per-salesperson assignment, with a configurable fallback.
Sales tax	Per location, US state-based, recorded on each order as charged.
What .env holds	Database credentials, mail transport, app key. Infrastructure only — verified.

The four surfaces

SURFACE	WHO USES IT	WHAT IT DOES
Admin panel	Managers, appraisers, accountants	Inventory, intake, review, orders, customers, settings, audit, roles, commissions, overrides
Point of Sale	Sales staff at the counter	Full-screen register: search, cart, discounts, split payment, returns, exchanges, receipts
Storefront	The public	Browse, filter, search, bag, checkout, customer accounts and order history
Background	Nobody — it just runs	Commission calculation, audit retention purge, receipt email

One codebase, one payment path. The register and the storefront both create orders through the same service and take money through the same gateway factory. A change to how money is handled happens once.

Getting started

From a fresh clone to a working till, and the order in which to set things up.

Installation

- 1

Install dependencies and build assets
composer install · npm install · npm run build
- 2

Point the application at a database
Copy .env.example to .env, set the database credentials, then run php artisan key:generate.
- 3

Create the schema and seed it
php artisan migrate --seed

Seeds six roles with 34 permissions, 45 settings at their defaults, four locations, the Stripe gateway row, 29 field rules across six categories, and placeholder step-up symbols.
- 4

Link storage and start
php artisan storage:link · php artisan serve
- 5

Run the queue worker
php artisan queue:work

Commission calculation runs here. Without a worker, sales still complete — commissions simply wait in the queue.

First-run configuration, in order

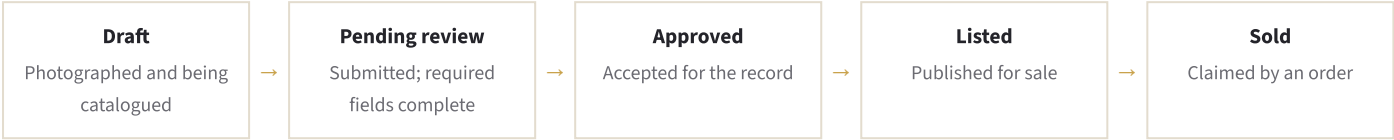
	WHERE	WHAT TO DO
1	Settings → General	Company name, timezone, receipt footer text.
2	Settings → Locations	Your real stores, each with its state and tax rate. Keep the Web location: online orders belong to it.
3	Settings → Payments	Paste the Stripe <i>test</i> keys, leave Test mode on. Live keys go in the separate live fields when you are ready.
4	Terminal	php artisan payments:verify — runs a real charge, refund and split against Stripe test mode and reports pass or fail.
5	Settings → Security	Which roles must use two-factor; session timeout; password policy; retention windows.
6	Settings → Field Rules	Adjust what each category requires before an item can be submitted.
7	Settings → Commission	The default plan used for any salesperson without an individual agreement.
8	Roles & Permissions	Create staff accounts, assign roles and home locations.

Do this before going live
Replace the seeded step-up answers. The installer seeds three symbols with the answers change-me-one, change-me-two, change-me-three. That challenge guards override approval and payment-key changes. A seeded secret is not a secret.

Items: intake, the colour system, and review

The centrepiece workflow. A piece goes from a pile of photographs to a listed item, with the record showing at a glance what is finished and what is not.

The journey of a piece



Approve and List are deliberately two separate actions. A piece can be approved for the record — catalogued, appraised, documented — without being published for sale. Listing is a second, explicit decision.

The colour-coded field system

Every field on an item record carries a status colour. The colours are configuration, held in a rules table and editable per category from Settings, so what a ring must have differs from what a brooch must have.

COLOUR	MEANING	BEHAVIOUR TODAY
● Red	Required, still empty	Blocks submission. Enforced on the server, not only in the browser.
● Yellow	Normally AI-suggested	An ordinary editable field, marked “AI suggestion available in a future update”.
● Green	Complete and valid	Computed the moment a valid value is entered. Never configured directly.
● Gray	Not applicable	Disabled for this category — chain length on a ring, for instance.
● Blue	Human override	Recorded on the item, so Phase 2's correction log has real data from day one.
○ Neutral	Optional, untouched	A sixth state added in build: an empty optional field shows no status, so an unfinished record cannot look complete.

What is recorded about an item

Identity & materials

SKU, title, subtitle, category and subcategory, brand or maker, style period, metal type and purity, weight in grams, measurements, hallmark text, plus any category-specific field a Super Admin adds later.

Condition & description

Condition and restoration notes, internal notes, customer description, and separate SEO, marketplace and social descriptions.

Gemstones

Stone type, shape, cut, colour, estimated carat weight, setting style, and which stone is the principal one. Several stones per piece.

Photographs

Each tagged by type — front, back, side, hallmark, gemstone, clasp, movement, signature, packaging, certificate — with one chosen as the hero shot used on the storefront.

Pricing (append-only history)

Acquisition cost, retail price, insurance value, negotiation floor, promotional price. A price change writes a new row rather than overwriting, so what a piece was previously offered at survives.

Stock & provenance

Which location holds it and in what state (in stock, reserved, sold, transferred); who created, submitted and approved it, and when.

A field that has no column still works. Because a Super Admin can invent a new field rule — say `movement_type` for watches — values for fields that have no database column are stored as item attributes. Adding a required field to a category needs no developer.

Speed, measured rather than assumed

The specification asks for photo upload through submit-for-review in well under five minutes. Each item records the elapsed time between creation and submission, and the review queue shows managers the **median** and **slowest** intake times, with the median turning red if it drifts above five minutes.

Moving stock between locations

A transfer is requested, approved by the **sending** location's manager, then completed on arrival. Stock does not move until completion, and an approved transfer marks the piece in transit so it cannot also be sold. The stock row moves; it is never duplicated.

The counter: sales, returns and exchanges

A full-screen register with the happy path kept to a minimum number of clicks. Target: a complete sale in under two minutes.

Taking a sale

- 1 Find the piece**
Search by SKU, title or maker, or scan a tag. Only listed, in-stock, unlocked items appear.
- 2 Build the sale**
Add pieces, plus any **custom line item** — ring sizing, repairs, a workshop service — which needs no inventory record. Discount a line by percentage or fixed amount.
- 3 Take payment**
Card, cash, or a split across several tenders. Reaching payment pre-fills a single card tender for the full amount, so the common sale is search → add → charge.
- 4 Finish**
Receipt on screen, printable on 80 mm thermal stock, and emailed when a customer address is on file. Change due is calculated and shown for cash.

Rules the register enforces

RULE	HOW IT BEHAVES
Tenders must equal the total	The Charge button stays disabled, and the server re-checks. A part-paid order can never be marked paid.
Discount ceiling	Above the configured percentage, the discount is refused unless the user holds the approval permission. Checked in code, not by hiding a button.
One-of-a-kind stock	The same piece cannot be added twice, and a piece sold elsewhere cannot be added at all.
The sold-once guarantee	Paying locks the stock row and flips it to sold inside the same transaction as the order and the payment.
Locked items	A piece under a sales or full lock cannot be sold, from any surface.

Returns and exchanges

Look up the original sale by order number, tick the items coming back, and choose a refund or an exchange.

- Refunds** return the line's discounted value *plus the tax charged on it*, capped at what remains refundable. Card refunds reverse through Stripe; cash is unrestricted. The piece goes back on the shelf and returns to listed.
- Exchanges** are a return and a replacement sale in one transaction. Only the difference changes hands — the customer pays the excess, or is refunded it if the replacement is cheaper.
- Outside the return window** (30 days by default, configurable) a refund is refused unless the user has override authority.

The public storefront

The same catalogue, the same stock, the same payment path — seen from the customer's side.

Browsing

Keyword search across title, maker, period, metal and description. Filters for category, period, metal and price range; sorting by newest or price. Facets are drawn only from pieces the public can actually see.

The piece

Image gallery with the hero shot first, customer description, specification table, gemstone details, price, and add-to-bag.

The visibility guarantee

Only pieces that are **listed**, **in stock** and **not locked** ever appear publicly. Draft, pending, approved-but-unlisted, archived, sold and reserved pieces are invisible — not merely hidden from the listing, but unreachable by guessing the URL. Seven hidden states are covered by tests.

Checkout

Guests may check out without an account. Card details are entered in Stripe's Payment Element, themed with the shop's own colours and typography, and confirmed by the browser directly with Stripe — **card data never reaches this application**. The server then re-reads the payment from Stripe before recording it, and refuses any amount that does not match the order. The browser is never trusted about whether money moved.

Sales tax follows the delivery address's state. A state with no store charges nothing rather than guessing.

Customer accounts

Customers sign in on their own separate account system — a customer session can never carry staff permissions. A guest who bought earlier and later registers with the same email **claims that record**, keeping their order history rather than starting a second one. POS and storefront share one customer record throughout, matched on email.

A piece sold at the counter disappears from web bags. The cart re-checks availability on every view, so a one-of-a-kind item sold in the shop cannot reach checkout online.

Payments

One gateway abstraction, keys held in the database, and every figure in whole cents.

How payments are structured

CAPABILITY	DETAIL
Card	Charged through Stripe. At the counter today this uses Stripe's test token — see the open items in Section 10.
Cash	Recorded with no gateway call; change due calculated from the amount tendered.
Split	Several tenders on one sale, in any mix. The parts must sum exactly to the total.
Refunds	Full or partial, drawn from card tenders first since cash is unrestricted.
Capture & void	Authorise, then settle for less or release entirely — backing the exchange flow.
Webhooks	Signed endpoint verified against the stored secret, so a refund issued in the Stripe dashboard is reflected here.

Three decisions worth knowing

Money moves before it is recorded. A gateway call cannot be rolled back, so charges happen outside the database transaction and records are written inside it. The reverse order would let a rolled-back transaction hide a real charge.

A failed split reverses what already succeeded. If the second tender is declined, the first card is refunded automatically. A customer is never left charged for a sale that did not complete.

Switching provider is a data change. Gateways are rows naming a driver class. Adding PayPal later is a row plus a class — the code that takes payment does not change.

Verifying the keys

Running `php artisan payments:verify` performs a real charge, a partial refund, the remainder, a card-and-cash split and a split refund against the configured gateway, reporting each. It refuses to run while Test mode is off, so it can never move real money. Run it after every key rotation.

Commission and payroll

Four plan structures, calculated away from the till, with California wage law shaping the design.

The four plan types

TYPE	BASIS	WORKED EXAMPLE
Sales-based	Flat percentage of the sale	5% of a \$1,000 sale = \$50.00
Profit-based	Percentage of margin over acquisition cost	20% of (\$1,000 – \$450) = \$110.00
Tiered	Rate rises at configured thresholds	4% below \$5,000; 6% of an \$8,000 sale = \$480.00
Split	A pool divided across several staff	5% of \$1,000 split 60/40 = \$30.00 / \$20.00

How it is calculated

- **Tax is never commissionable.** Every plan works from subtotal less discount — sales tax was never the shop's money.
- **Discounts reduce the commissionable value,** so a discounted sale earns proportionally less.
- **The logged-in salesperson earns it.** Web sales attribute to nobody.
- **It runs in the background,** queued after the sale commits. Commission arithmetic never sits between a customer's card and their receipt, and never runs for a sale that rolled back.
- **A salesperson without an individual agreement** falls back to the default plan configured in Settings.

The rounding problem, solved deliberately

The classic split-commission bug is rounding: three people sharing a pool that will not divide evenly, and a cent goes missing. Shares are floored, then the remainder is handed out one cent at a time to the largest fractional parts. The parts always sum *exactly* to the pool — checked across forty pool-and-share combinations, including deliberately awkward ones.

California compliance, built in

STATUTE	WHAT THE SYSTEM DOES
§2751 Written agreements	Each staff assignment stores the agreed terms — text or an uploaded document — alongside the numeric plan. The plan alone is not the agreement, and the system distinguishes the two.
§221 No unlawful deductions	Nothing deducts from an earned commission. A refund leaves the record standing. Any reduction must go through an override — reasoned, approved and logged.
§204 Wage timing	Approval and payment are recorded as separate, timestamped steps, so wage timing is auditable.

Reporting and payroll export

Daily, weekly or monthly, filtered by salesperson, location and status. Staff see only their own figures; managers and accountants see everyone's. Export to CSV is a separate permission.

The export's **column mapping is configurable**, because the target payroll system's format is not yet known. Changing which columns appear, and what each contains, is a settings change. A column pointing at something the system does not recognise exports a visible marker rather than a silent blank.

Security, permissions and the audit trail

Six roles, thirty-four granular permissions, two-factor authentication nobody can skip, and a record of everything.

The six roles

ROLE	PERMS	REMIT
Super Admin	34	Everything, including settings, roles and overrides
Store Manager	21	Inventory, orders, approvals, discounts above the staff ceiling, audit, commission reports
Sales Staff	11	The register, creating and editing inventory subject to approval, own commission
Inventory Specialist	7	Create, edit and approve inventory for listing. No sales.
Accountant	9	Financial and commission reports, payment records, payroll export, audit. No inventory edits, no sales.
Customer Service	7	View orders and customers, process returns. No discounting, no pricing edits.

Nothing in the code checks a role by name — only permissions — so a Super Admin can compose a custom role from the Roles & Permissions page and it behaves exactly like a built-in one.

Multi-factor authentication

Which roles must use it is a setting. A user in one of those roles is sent to enrolment on first sign-in and challenged on every sign-in thereafter. Enrolment issues ten single-use recovery codes, shown once. **No role is exempt, including Super Admin** — the requirement is enforced across every authenticated page, not route by route.

The authenticator secret is encrypted at rest; recovery codes are additionally hashed, so a database leak yields nothing usable. The enrolment QR code is drawn on the server — the secret never reaches a third party.

The audit trail

Every create, update and delete on products, orders, payments, settings, users, customers, locations, transfers, overrides and locks is recorded automatically — who, what, when, before and after, with IP address and browser. Logging is attached to the models themselves, so a new code path cannot forget to call it.

- **Secrets are masked.** A Stripe key rotation is recorded as a change, with the value redacted. A gateway switch stays readable.
- **Non-model events too:** sign-in, failed sign-in, MFA passes and failures, step-up attempts, webhook rejections.
- **Retention is split by category.** General activity keeps for two years; financial records keep for seven, reflecting IRS recordkeeping. Both are configurable, and a scheduled job applies them nightly.

Overrides

Deliberately high-friction, because this exists for fraud prevention. A reason is **required**, and nothing takes effect until someone with approval authority approves it. A Super Admin overriding their own request still goes through both steps, and the trail records that they approved their own request.

Inventory locks

LOCK	BLOCKS	LEAVES WORKING
Full	Sale, edit, rental, storefront display	Nothing — the piece is frozen
Sales	Sale and rental	Editing: re-photography, re-appraisal
Edit	Editing the record	Selling — for use mid-dispute
View	Storefront and POS visibility	Internal access, for a legal hold

Rental	Rental only	Everything else (Phase 2 relevance)
--------	-------------	-------------------------------------

Locks are enforced in the services that sell, edit and transfer — so the register, the storefront and any future integration are all covered by one check. The reason is free text: valuation hold, repair, consignment dispute, legal hold.

The step-up challenge

High-risk actions — approving an override, changing payment keys, disabling MFA enforcement — require a second challenge: two or three symbols whose answers only trusted staff know. Both the symbol set and the list of actions it guards are configurable, and every attempt, pass or fail, is logged.

This is a second factor with a house style. The security comes from the answers being a shared secret, not from the symbols themselves — so answers are hashed, and the mechanism can be swapped for a standard authenticator step-up with a configuration change.

Against the specification

Every acceptance criterion in the brief, and where it stands in the delivered build.

DEFINITION OF DONE	STATUS	EVIDENCE
Every Super-Admin setting lives in Settings, not <code>.env</code>	MET	45 settings across 7 groups; <code>.env</code> holds infrastructure only
Stripe works for card, cash, split, refunds, exchanges	MET	Verified against live Stripe test keys, not only mocks
Zero hardcoded AI vendor references	MET	Enforced by a test scanning the whole codebase
All six roles enforce permissions at the policy layer	MET	Nine protected actions × six roles asserted as one matrix
MFA and audit cannot be bypassed by any role	MET	Middleware across every authenticated page; Super Admin included
Location scoping restricts cross-location visibility	MET	Tested against a user <i>without</i> the cross-location permission
Photo upload → submit for review under five minutes	MET	Measured per item; median and slowest shown to managers
A POS sale completes in under two minutes	MET	Tested, and driven end to end through the browser
All Phase 2 flags exist, default off, visible in Settings	MET	Three flags, each with tables and a nav placeholder
No arbitrary styling values anywhere	MET	Design tokens only; off-system values fail the build

Deliberate departures from the brief

Each was raised at the time and agreed. None changes what the system does.

THE BRIEF SAID	WHAT WAS BUILT, AND WHY
Laravel 11	Laravel 13. Version 11 is past security support and shipped with three known advisories, including a high-severity one, fixed only in later versions.
Redis and Horizon	Database queue. Redis is not installed on this host; the brief permits the fallback.
Tokens in <code>tailwind.config.js</code>	CSS-based tokens. That file does not exist in the current styling framework; the discipline it protects is enforced in the equivalent place.
A “Reveal” button for the Stripe secret	Not built. It contradicted the rule that secrets are never echoed back. Keys are write-only after saving; rotation replaces them.
Five field colours	Six. Green is computed from a value, so an empty optional field needed a neutral state — otherwise an untouched record looked finished.
The design's role names and lock names	The specification's. Confirmed with you; the design's lock names became the free-text reason.

Bugs found by running the system, not by testing it

Recorded because each would otherwise have reached production.

- **Sales tax rounding.** The tax column was too narrow for a real US rate: New York's 8.875% became 8.88%, overcharging roughly three cents in every hundred dollars, on every sale.
- **Online orders never completing.** Web orders belong to a Web location while stock sits in a physical shop, so payment succeeded but the order stayed unpaid and the piece stayed for sale.
- **Invisible text** in the sign-in and register fields — white on white.
- **A test double that was too generous,** reporting unconfirmed card payments as successful. Correcting it turned a quietly passing test into a real one.

Open items and what comes next

What still needs doing before the counter opens, and what Phase 2 is already prepared for.

Before going live

ITEM	PRIORITY	DETAIL
Replace step-up answers	BLOCKING	Seeded placeholders guard override approval and payment-key changes.
Card entry at the counter	BLOCKING	The register currently passes Stripe's test token. The themed card form is built and working on the storefront; the same component needs mounting on the register. No card reader is in scope, as agreed.
Mailgun credentials	NEEDED	Blank, so receipt emails will not send. Everything else about receipts works.
Live Stripe keys	NEEDED	Separate fields already exist; switch Test mode off when ready.
Image library	OPTIONAL	Without it, photographs are stored at full size and no thumbnails are generated.
Webhook endpoint	OPTIONAL	Needs a public URL. Only affects refunds issued from the Stripe dashboard.
Queue worker & scheduler	NEEDED	Must run continuously in production for commissions and nightly retention.

Phase 2, already seamed

Each is switched off, with its tables and integration points already in place, so activation means building behaviour rather than reshaping the system.

FEATURE	WHAT ALREADY EXISTS
AI assistance Photo analysis, descriptions, pricing, search	Provider interfaces, a results store, and a correction log that captures what a human changed a suggestion to. The yellow/green/blue field colours already carry the meaning AI will need. The codebase contains no vendor references at all — providers are data rows.
Rental services	Agreement and claim tables; the rental lock type already exists on inventory.
Salesperson storefronts	Storefront and inventory-request tables; commission already supports split attribution, which is how a referred sale will be shared.
Quarterly audit	Report and exception tables; every stock movement is already logged, which is the data a reconciliation reads.

Where the detail lives

Every business rule you answered during the build, and every place the delivered system departs from the brief, is recorded in docs/DECISIONS.md in the repository — with the reasoning, so nothing has to be reconstructed from memory.

The build history is twelve module commits plus two fixes, each with its own description of what changed and why.